

MTC-242-MN-P: All Slips Solutions

S.Y. B.Sc. (Computer Science) — Mathematics Practical Examination (sem III)

Solutions by Raj...

Slip No. 1

Q.1. Attempt any one of the following .

1) Regula-Falsi: root of $x^3 - 5x - 9 = 0$ (bracket [2,3]) correct to 5 decimals or 15 iterations

```
def f(x):
    return x**3 - 5*x - 9

a, b = 2, 3
tol = 0.5e-5
max_iter = 15

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 2.785714, f(c) = -1.310860
Iteration 2: c = 2.850875, f(c) = -0.083923
Iteration 3: c = 2.854933, f(c) = -0.005125
Iteration 4: c = 2.855180, f(c) = -0.000312
Iteration 5: c = 2.855196, f(c) = -0.000019
Iteration 6: c = 2.855196, f(c) = -0.000001
```

Approximate root = 2.85520 (after 6 iterations)

2) Newton's forward difference formula: evaluate $f(1.7)$

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([40, 60, 65, 50, 18], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 1.7
```

```

p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(1.7) ≈ 55.13820
```

Q.2. Attempt any two of the following .

1) Write a python program to estimate the value of the integral $\int_{-3}^3 x^2 dx$, using Trapezoidal rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$.

```

import numpy as np

def f(x):
    return x**2

a, b = -3, 3
n = 12
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	-3.0000	9.0000
1	-2.5000	6.2500
2	-2.0000	4.0000
3	-1.5000	2.2500
4	-1.0000	1.0000
5	-0.5000	0.2500
6	0.0000	0.0000
7	0.5000	0.2500
8	1.0000	1.0000
9	1.5000	2.2500
10	2.0000	4.0000
11	2.5000	6.2500
12	3.0000	9.0000

```
Approximate integral = 18.25000
```

2) Write Python program to find $f(10)$, using Lagrange's interpolation formula for the data: $f(5) = 12$, $f(6) = 13$, $f(9) = 14$, $f(11) = 16$.

```
import numpy as np

x = np.array([5, 6, 9, 11], dtype=float)
y = np.array([12, 13, 14, 16], dtype=float)
n = len(x)

xp = 10
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(10) ≈ 14.66667
```

3) Write a python program to find $y(2.2)$, for the differential equation $\frac{dy}{dx}$ Modified Method where $y(2) = 1$ take $h = 0.1$.

```
def f(x, y):
    return -x * y**2

x0, y0 = 2, 1
h = 0.1
x_end = 2.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 2.1000, y = 0.83280
```

```
x = 2.2000, y = 0.70804
```

Slip No. 2

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of the equation $x^3 - 10x^2 + 5 = 0$ in $[0, 1]$, using False Position method the program should terminate when relative error is of order 10^{-6} or perform 25 number of iterations whichever occurs earlier.

```
def f(x):
    return x**3 - 10*x**2 + 5

a, b = 0, 1
tol = 1e-6
max_iter = 25

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 0.555556, f(c) = 2.085048
Iteration 2: c = 0.707845, f(c) = 0.344218
Iteration 3: c = 0.730994, f(c) = 0.047085
Iteration 4: c = 0.734124, f(c) = 0.006270
Iteration 5: c = 0.734540, f(c) = 0.000832
Iteration 6: c = 0.734595, f(c) = 0.000110
Iteration 7: c = 0.734602, f(c) = 0.000015
Iteration 8: c = 0.734603, f(c) = 0.000002
Iteration 9: c = 0.734603, f(c) = 0.000000
```

```
Approximate root = 0.73460 (after 9 iterations)
```

2) Write a Python program to evaluate $f(1.7)$ by Newton's backward difference formula using the given data.

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([40, 60, 65, 50, 18], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 1.7
```

```

p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
    fact *= j
    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(1.7) ≈ 55.13820
```

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral: $\int_1^7 (1/x) dx$, $n = 12$ using Simpson's (1/3)rd rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3 \dots 12$

```

import numpy as np

def f(x):
    return 1 / x

a, b = 1, 7
n = 12          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	1.0000	1.0000
1	1.5000	0.6667
2	2.0000	0.5000
3	2.5000	0.4000
4	3.0000	0.3333
5	3.5000	0.2857
6	4.0000	0.2500
7	4.5000	0.2222
8	5.0000	0.2000
9	5.5000	0.1818
10	6.0000	0.1667
11	6.5000	0.1538
12	7.0000	0.1429

```
Approximate integral = 1.94732
```

2) Write Python program to prepare divided difference table for the data:

X	1	2	5	7
Y = f(x)	6	9	30	54

```
import numpy as np
import pandas as pd

x = np.array([1, 2, 5, 7], dtype=float)
y = np.array([6, 9, 30, 54], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)
```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD	Order-3 DD
0	1.0	6.0	3.0	1.0	0.0
1	2.0	9.0	7.0	1.0	
2	5.0	30.0	12.0		
3	7.0	54.0			

3) Write a python program to solve the differential equation, $dy/dx = (x+y)$ for $x = 0$ to $x = 0.3$ by RungeKutta second order Method where $y(0) = 1$ take $h = 0.1$.

```
def f(x, y):
    return x + y

x0, y0 = 0, 1
h = 0.1
x_end = 0.3

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)
    y = y + (k1 + k2) / 2
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.1000, y = 1.11000
x = 0.2000, y = 1.24205
x = 0.3000, y = 1.39847
```

Slip No. 3

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of equation $3x - \cos(x) - 1 = 0$, using Bisection method correct up to three decimal places. Plot the graph of function $y = f(x)$, also plot the roots obtained.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return 3*x - np.cos(x) - 1

a, b = 0, 1
tol = 0.5e-3
max_iter = 50

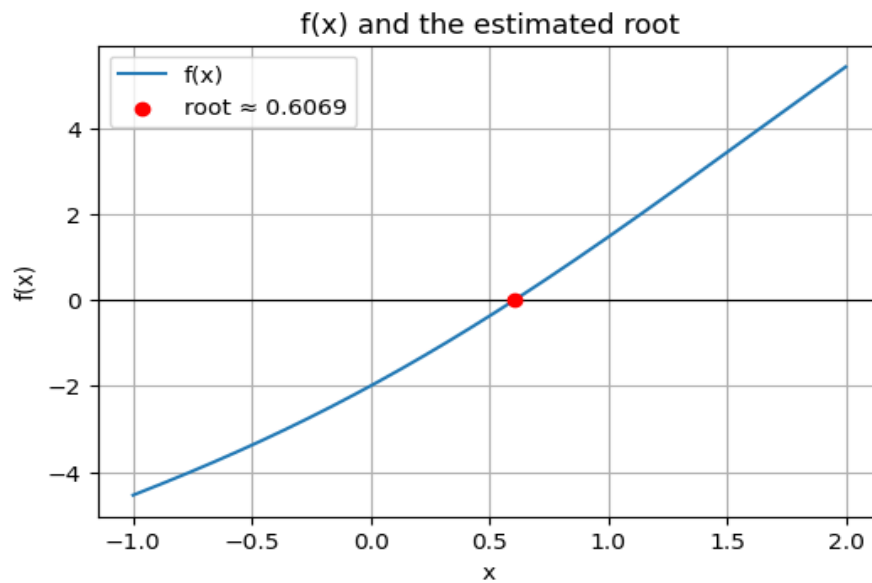
roots_found = []
c = a
for i in range(1, max_iter + 1):
    c = (a + b) / 2
    roots_found.append(c)
    print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
    if abs(f(c)) < tol or (b - a) / 2 < tol:
        break
    if f(a) * f(c) < 0:
        b = c
    else:
        a = c
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")

xs = np.linspace(-1, 2, 400)
ys = [f(v) for v in xs]
plt.figure(figsize=(6, 4))
plt.plot(xs, ys, label="f(x)")
plt.axhline(0, color='black', linewidth=0.8)
plt.scatter([c], [f(c)], color='red', zorder=5, label=f"root ≈ {c:.4f}")
plt.title("f(x) and the estimated root")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.legend(); plt.grid(True)
plt.show()
```

Output:

```
Iteration 1: c = 0.500000, f(c) = -0.377583
Iteration 2: c = 0.750000, f(c) = 0.518311
Iteration 3: c = 0.625000, f(c) = 0.064037
Iteration 4: c = 0.562500, f(c) = -0.158424
Iteration 5: c = 0.593750, f(c) = -0.047598
Iteration 6: c = 0.609375, f(c) = 0.008119
Iteration 7: c = 0.601562, f(c) = -0.019765
Iteration 8: c = 0.605469, f(c) = -0.005829
Iteration 9: c = 0.607422, f(c) = 0.001143
Iteration 10: c = 0.606445, f(c) = -0.002343
Iteration 11: c = 0.606934, f(c) = -0.000600
```

Approximate root = 0.60693 (after 11 iterations)



2) Write a Python program to evaluate $f(1.7)$ by Newton's divided difference formula using the given data.

```
import numpy as np

x = np.array([1, 2, 5, 7], dtype=float)
y = np.array([6, 9, 30, 54], dtype=float)
n = len(x)

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 1.7
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(1.7) \approx 7.89000$

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral $\int_0^{\pi} \sin(x) dx$, using Trapezoidal rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$.

```
import numpy as np

def f(x):
    return np.sin(x)

a, b = 0, np.pi
```

```

n = 12
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	0.0000	0.0000
1	0.2618	0.2588
2	0.5236	0.5000
3	0.7854	0.7071
4	1.0472	0.8660
5	1.3090	0.9659
6	1.5708	1.0000
7	1.8326	0.9659
8	2.0944	0.8660
9	2.3562	0.7071
10	2.6180	0.5000
11	2.8798	0.2588
12	3.1416	0.0000

Approximate integral = 1.98856

2) Write Python program to prepare forward difference table for the given data:

X	1	2	3	4	5
Y = f(x)	40	60	65	50	18

```

import numpy as np
import pandas as pd

x = [1, 2, 3, 4, 5]
y = [40, 60, 65, 50, 18]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = D[i+1, j-1] - D[i, j-1]

columns = ['y'] + [f'Δ{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Δ{j}y'] = D[i, j]
df.insert(0, 'x', x)

```

```
print("Newton's Forward Difference Table:\n")
print(df)
```

Output:

Newton's Forward Difference Table:

	x	y	Δ^1y	Δ^2y	Δ^3y	Δ^4y
0	1	40	20.0	-15.0	-5.0	8.0
1	2	60	5.0	-20.0	3.0	
2	3	65	-15.0	-17.0		
3	4	50	-32.0			
4	5	18				

3) Write a python program to find $y(0.1)$ and $y(0.2)$, for the differential equation $dy/dx = y-x$ by RungeKutta Fourth order Method where $y(0) = 2$ take $h = 0.1$.

```
def f(x, y):
    return y - x

x0, y0 = 0, 2
h = 0.1
x_end = 0.2
n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.1000, y = 2.20517
x = 0.2000, y = 2.42140
```

Slip No. 4

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of algebraic equation $x^3 - 2x + 1 = 0$ in interval $[-1, 0]$, using Newton Raphson method correct up to five decimal or perform 15 number of iterations whichever occurs earlier.

```
import sympy as sp

x = sp.symbols('x')
f_expr = x**3 - 2*x + 1
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = -1.0
tol = 0.5e-5
max_iter = 15

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break
print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: x = -3.000000
Iteration 2: x = -2.200000
Iteration 3: x = -1.780831
Iteration 4: x = -1.636303
Iteration 5: x = -1.618305
Iteration 6: x = -1.618034
Iteration 7: x = -1.618034
```

Approximate root = -1.61803 (after 7 iterations)

2) Write a Python program to evaluate $f(97)$ by Newton's backward difference formula using the given data .

X	80	85	90	95	100
Y = f(x)	5026	5674	6362	7088	7854

```
import numpy as np

x = np.array([80, 85, 90, 95, 100], dtype=float)
y = np.array([5026, 5674, 6362, 7088, 7854], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
```

```

for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 97
p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
    fact *= j
    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

f(97) ≈ 7389.35360

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral: $\int_0^1 x^*e^x dx$, using Simpson's 1/3 rd rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$

```

import numpy as np

def f(x):
    return x * np.exp(x)

a, b = 0, 1
n = 12          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	0.0000	0.0000
1	0.0833	0.0906
2	0.1667	0.1969
3	0.2500	0.3210
4	0.3333	0.4652
5	0.4167	0.6320
6	0.5000	0.8244
7	0.5833	1.0453
8	0.6667	1.2985
9	0.7500	1.5878
10	0.8333	1.9175
11	0.9167	2.2925

```
12      1.0000      2.7183
```

```
Approximate integral = 1.00000
```

2) Write Python program to find $f(4)$, using Lagrange's interpolation formula for the data: $f(1) = 6$, $f(2) = 9$, $f(5) = 30$, $f(7) = 54$.

```
import numpy as np

x = np.array([1, 2, 5, 7], dtype=float)
y = np.array([6, 9, 30, 54], dtype=float)
n = len(x)

xp = 4
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(4) ≈ 21.00000
```

3) Write a python program to find $y(1)$, for the differential equation $dy/dx = -2*x*y^2$, by Euler's Modified Method where $y(0) = 1$ take $h = 0.2$.

```
def f(x, y):
    return -2 * x * y**2

x0, y0 = 0, 1
h = 0.2
x_end = 1

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.2000, y = 0.96000
x = 0.4000, y = 0.86030
x = 0.6000, y = 0.73504
x = 0.8000, y = 0.61157
x = 1.0000, y = 0.50334
```

Slip No. 5

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of the equation $X \log_{10}(x) - 1.2 = 0$, using Bisection method the program should terminate when relative error is of order 10^{-4} or perform 25 number of iterations whichever occurs earlier.

```
def f(x):
    return x * np.log10(x) - 1.2

a, b = 2, 3
tol = 1e-4
max_iter = 25

if f(a) * f(b) > 0:
    print("f(a) and f(b) have the same sign; choose a different interval.")
else:
    c = a
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
        if abs(f(c)) < tol or (b - a) / 2 < tol:
            break
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 2.500000, f(c) = -0.205150
Iteration 2: c = 2.750000, f(c) = 0.008165
Iteration 3: c = 2.625000, f(c) = -0.099786
Iteration 4: c = 2.687500, f(c) = -0.046126
Iteration 5: c = 2.718750, f(c) = -0.019059
Iteration 6: c = 2.734375, f(c) = -0.005466
Iteration 7: c = 2.742188, f(c) = 0.001345
Iteration 8: c = 2.738281, f(c) = -0.002062
Iteration 9: c = 2.740234, f(c) = -0.000359
Iteration 10: c = 2.741211, f(c) = 0.000493
Iteration 11: c = 2.740723, f(c) = 0.000067
```

Approximate root = 2.74072 (after 11 iterations)

2) Write a Python program to evaluate $f(1.7)$ by Newton's divided difference formula using the given data.

X	0	1	2	5
Y = f(x)	5	13	22	129

```
import numpy as np

x = np.array([0, 1, 2, 5], dtype=float)
y = np.array([5, 13, 22, 129], dtype=float)
n = len(x)

diff = np.zeros((n, n))
```

```

diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 1.7
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

f(1.7) ≈ 18.75470

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral: $\int_2^5 (x^2 - 2x + 1) dx$, using Trapezoidal rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$.

```

import numpy as np

def f(x):
    return x**2 - 2*x + 1

a, b = 2, 5
n = 12
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	2.0000	1.0000
1	2.2500	1.5625
2	2.5000	2.2500
3	2.7500	3.0625
4	3.0000	4.0000
5	3.2500	5.0625
6	3.5000	6.2500
7	3.7500	7.5625
8	4.0000	9.0000
9	4.2500	10.5625
10	4.5000	12.2500
11	4.7500	14.0625
12	5.0000	16.0000

Approximate integral = 21.03125

2) Write Python program to prepare backward difference table for the data: $f(1) = 40$, $f(2) = 60$, $f(3) = 65$, $f(4) = 50$, $f(5) = 18$

```
import numpy as np
import pandas as pd

x = [1, 2, 3, 4, 5]
y = [40, 60, 65, 50, 18]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(j, n):
        D[i, j] = D[i, j-1] - D[i-1, j-1]

columns = ['y'] + [f'∇{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(j, n):
        df.loc[i, f'∇{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Backward Difference Table:\n")
print(df)
```

Output:

Newton's Backward Difference Table:

	x	y	∇ ¹ y	∇ ² y	∇ ³ y	∇ ⁴ y
0	1	40				
1	2	60	20.0			
2	3	65	5.0	-15.0		
3	4	50	-15.0	-20.0	-5.0	
4	5	18	-32.0	-17.0	3.0	8.0

3) Write a python program to find $y(0.6)$, for the differential equation $dy/dx = 1 + y^2$, by RungeKutta second order Method where $y(0) = 0$ take $h = 0.2$.

```
def f(x, y):
    return 1 + y**2

x0, y0 = 0, 0
h = 0.2
x_end = 0.6

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)
    y = y + (k1 + k2) / 2
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.2000, y = 0.20400
x = 0.4000, y = 0.42516
x = 0.6000, y = 0.68697
```

Slip No. 6

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of the algebraic equation $x^3 - x^2 - 2 = 0$ between 1 and 2, using Newton Raphson method. The program should terminate when relative error is of order 0.000001 or perform n number of iterations whichever occurs earlier.

```
import sympy as sp

x = sp.symbols('x')
f_expr = x**3 - x**2 - 2
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = 1.5
tol = 1e-6
max_iter = 20

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break
print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: x = 1.733333
Iteration 2: x = 1.696688
Iteration 3: x = 1.695622
Iteration 4: x = 1.695621
```

```
Approximate root = 1.69562 (after 4 iterations)
```

2) Write a Python program to evaluate $f(1.8)$ by Newton's forward difference formula using the given data.

X	1.5	1.7	1.9	2.1	2.3
Y = f(x)	1494	1691	1888	2084	2279

```
import numpy as np

x = np.array([1.5, 1.7, 1.9, 2.1, 2.3], dtype=float)
y = np.array([1494, 1691, 1888, 2084, 2279], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 1.8
```

```

p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(1.8) ≈ 1789.58594
```

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral: $\int_1^2 (x^2 - 10) dx$, using Simpson's 3/8 th rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$.

```

import numpy as np

def f(x):
    return (x**2 - 10)**2

a, b = 1, 2
n = 12          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	1.0000	81.0000
1	1.0833	77.9051
2	1.1667	74.6304
3	1.2500	71.1914
4	1.3333	67.6049
5	1.4167	63.8889
6	1.5000	60.0625
7	1.5833	56.1459
8	1.6667	52.1605
9	1.7500	48.1289
10	1.8333	44.0748
11	1.9167	40.0232
12	2.0000	36.0000

```
Approximate integral = 59.53335
```

2) Write Python program to prepare divided difference table for the data:

X	2	4	9	10
Y = f(x)	4	56	711	980

```
import numpy as np
import pandas as pd

x = np.array([2, 4, 9, 10], dtype=float)
y = np.array([4, 56, 711, 980], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)
```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD	Order-3 DD
0	2.0	4.0	26.0	15.0	1.0
1	4.0	56.0	131.0	23.0	
2	9.0	711.0	269.0		
3	10.0	980.0			

3) Write a python program to find y (1.4), for the differential equation $dy/dx = y^2 + x^2$, $y(1)=0$, $h=0.2$

```
def f(x, y):
    return y**2 + x**2

x0, y0 = 1, 0
h = 0.2
x_end = 1.4

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 1.2000, y = 0.24633
x = 1.4000, y = 0.62275
```

Slip No. 7

Q.1. Attempt any one of the following

1) Write a Python program to estimate a root of the equation $x^3 - 9x + 1 = 0$, using Regula-Falsi method correct up to five decimal places Or 15 number of iterations whichever occurs earlier.

```
def f(x):
    return x**3 - 9*x + 1

a, b = 0, 1
tol = 0.5e-5
max_iter = 15

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 0.125000, f(c) = -0.123047
Iteration 2: c = 0.111304, f(c) = -0.000360
Iteration 3: c = 0.111264, f(c) = -0.000001
Iteration 4: c = 0.111264, f(c) = -0.000000
```

```
Approximate root = 0.11126 (after 4 iterations)
```

2) Write a Python program to evaluate $f(48)$ by Newton's backward difference formula using the given data.

X	30	35	40	45	50
Y = f(x)	0.5	0.5736	0.5736	0.7071	0.7660

```
import numpy as np

x = np.array([30, 35, 40, 45, 50], dtype=float)
y = np.array([0.5, 0.5736, 0.5736, 0.7071, 0.766], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
```

```

        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 48
p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
    fact *= j
    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

f(48) ≈ 0.78198

Q.2. Attempt any two of the following

1) Write a python program to estimate the value of the integral: $\int_0^5 (1 + x^3) dx$, using Simpson's 3/8 rule, use $n = 12$. Print all values of x_i and $y_i = f(x_i)$, $i = 0, 1, 2, 3, \dots, 12$.

```

import numpy as np

def f(x):
    return 1 + x**3

a, b = 0, 5
n = 12          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	0.0000	1.0000
1	0.4167	1.0723
2	0.8333	1.5787
3	1.2500	2.9531
4	1.6667	5.6296
5	2.0833	10.0422
6	2.5000	16.6250
7	2.9167	25.8119
8	3.3333	38.0370
9	3.7500	53.7344
10	4.1667	73.3380
11	4.5833	97.2818
12	5.0000	126.0000

Approximate integral = 161.25000

2) Write Python program to find $f(4)$, using Lagrange's interpolation formula for the data: $f(1) = 6$, $f(2) = 9$, $f(5) = 30$, $f(7) = 54$.

```
import numpy as np

x = np.array([1, 2, 5, 7], dtype=float)
y = np.array([6, 9, 30, 54], dtype=float)
n = len(x)

xp = 4
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(4) \approx 21.00000$

3) Write a python program to find $y(1)$, for the differential equation, $dy/dx = 2 - x*y^2$, by Euler's Method where $y(0) = 1$ take $h = 0.2$.

```
def f(x, y):
    return 2 - x * y**2

x0, y0 = 0, 1
h = 0.2
x_end = 1

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y = y + h * f(x, y)
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

$x = 0.2000, y = 1.40000$
 $x = 0.4000, y = 1.72160$
 $x = 0.6000, y = 1.88449$
 $x = 0.8000, y = 1.85833$
 $x = 1.0000, y = 1.70579$

Slip No. 8

Q.1. Attempt any one of the following

1) Newton's divided difference formula: evaluate $f(2)$ (Full Q see in slip)

```
import numpy as np

x = np.array([-1, 0, 3, 6, 7], dtype=float)
y = np.array([3, -6, 39, 822, 1611], dtype=float)
n = len(x)

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 2
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(2) \approx 6.00000$

2) Bisection: root of $x - \cos(x) = 0$ in $[0,1]$, correct to 4 decimals or 20 iterations (Full Q see in slip)

```
def f(x):
    return x - np.cos(x)

a, b = 0, 1
tol = 0.5e-4
max_iter = 20

if f(a) * f(b) > 0:
    print("f(a) and f(b) have the same sign; choose a different interval.")
else:
    c = a
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
        if abs(f(c)) < tol or (b - a) / 2 < tol:
            break
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

Iteration 1: c = 0.500000, f(c) = -0.377583
Iteration 2: c = 0.750000, f(c) = 0.018311
Iteration 3: c = 0.625000, f(c) = -0.185963
Iteration 4: c = 0.687500, f(c) = -0.085335
Iteration 5: c = 0.718750, f(c) = -0.033879
Iteration 6: c = 0.734375, f(c) = -0.007875

```

Iteration 7: c = 0.742188, f(c) = 0.005196
Iteration 8: c = 0.738281, f(c) = -0.001345
Iteration 9: c = 0.740234, f(c) = 0.001924
Iteration 10: c = 0.739258, f(c) = 0.000289
Iteration 11: c = 0.738770, f(c) = -0.000528
Iteration 12: c = 0.739014, f(c) = -0.000120
Iteration 13: c = 0.739136, f(c) = 0.000085
Iteration 14: c = 0.739075, f(c) = -0.000017

```

Approximate root = 0.73907 (after 14 iterations)

Q.2. Attempt any two of the following

1) Newton's forward difference table (Full Q see in slip)

```

import numpy as np
import pandas as pd

x = [10, 11, 12, 13, 14, 15, 16]
y = [18, 25, 40, 70, 90, 100, 110]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = D[i+1, j-1] - D[i, j-1]

columns = ['y'] + [f'Δ^{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Δ^{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Forward Difference Table:\n")
print(df)

```

Output:

Newton's Forward Difference Table:

	x	y	Δ ¹ y	Δ ² y	Δ ³ y	Δ ⁴ y	Δ ⁵ y	Δ ⁶ y
0	10	18	7.0	8.0	7.0	-32.0	57.0	-72.0
1	11	25	15.0	15.0	-25.0	25.0	-15.0	
2	12	40	30.0	-10.0	0.0	10.0		
3	13	70	20.0	-10.0	10.0			
4	14	90	10.0	0.0				
5	15	100	10.0					
6	16	110						

2) RK2 (Heun) Method: find y(0.1) and y(0.2) for dy/dx = y - x, y(0)=2, h=0.1 (Full Q see in slip)

```

def f(x, y):
    return y - x

x0, y0 = 0, 2
h = 0.1
x_end = 0.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)

```

```

k2 = h * f(x + h, y + k1)
y = y + (k1 + k2) / 2
x = x + h
print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.1000, y = 2.20500
x = 0.2000, y = 2.42103

```

3) Trapezoidal rule: $\int_0^1 (4x - 3x^2) dx$, $n = 10$ (Full Q see in slip)

```

import numpy as np

def f(x):
    return 4*x - 3*x**2

a, b = 0, 1
n = 10
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	0.0000	0.0000
1	0.1000	0.3700
2	0.2000	0.6800
3	0.3000	0.9300
4	0.4000	1.1200
5	0.5000	1.2500
6	0.6000	1.3200
7	0.7000	1.3300
8	0.8000	1.2800
9	0.9000	1.1700
10	1.0000	1.0000

Approximate integral = 0.99500

Slip No. 9

Q.1. Attempt any one of the following

1) Regula-Falsi: root of $x^3 - 4x + 1 = 0$ in $[0,1]$, $\text{rel.err} < 1e-5$ or 25 iterations (Full Q see in slip)

```
def f(x):
    return x**3 - 4*x + 1

a, b = 0, 1
tol = 1e-5
max_iter = 25

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 0.333333, f(c) = -0.296296
Iteration 2: c = 0.257143, f(c) = -0.011569
Iteration 3: c = 0.254202, f(c) = -0.000382
Iteration 4: c = 0.254105, f(c) = -0.000013
Iteration 5: c = 0.254102, f(c) = -0.000000
Iteration 6: c = 0.254102, f(c) = -0.000000
```

Approximate root = 0.25410 (after 6 iterations)

2) Newton's backward interpolation formula: evaluate $f(8)$ (Full Q see in slip)

```
import numpy as np

x = np.array([1, 3, 5, 7, 9], dtype=float)
y = np.array([8, 12, 21, 36, 62], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 8
p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
```

```

    fact *= j
    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(8) ≈ 47.15625
```

Q.2. Attempt any two of the following

1) Lagrange interpolation: find $f(2)$ from $f(0)=12$, $f(3)=6$, $f(4)=8$. (Full Q see in slip)

```

import numpy as np

x = np.array([0, 3, 4], dtype=float)
y = np.array([12, 6, 8], dtype=float)
n = len(x)

xp = 2
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(2) ≈ 6.00000
```

2) Euler's Modified Method: find $y(0.05)$ and $y(0.1)$ for $dy/dx = x^2 + y$, $y(0)=1$, $h=0.05$. (Full Q see in slip)

```

def f(x, y):
    return x**2 + y

x0, y0 = 0, 1
h = 0.05
x_end = 0.1

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.0500, y = 1.05131
x = 0.1000, y = 1.10551

```

3) Simpson's 1/3 rule: $\int_0^1 1/(1+x) dx$, $n = 8$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 1 / (1 + x)

a, b = 0, 1
n = 8          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	1.0000
1	0.1250	0.8889
2	0.2500	0.8000
3	0.3750	0.7273
4	0.5000	0.6667
5	0.6250	0.6154
6	0.7500	0.5714
7	0.8750	0.5333
8	1.0000	0.5000

Approximate integral = 0.69315

Slip No. 10

Q.1. Attempt any one of the following

1) Newton's forward interpolation formula: evaluate $f(1)$. (Full Q see in slip)

```
import numpy as np

x = np.array([2, 4, 6, 8, 10], dtype=float)
y = np.array([5, 10, 17, 29, 50], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 1
p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(1) \approx 2.58594$

2) Newton-Raphson: root of $2x^3 - 3x - 6 = 0$, 5 decimals or 15 iterations . (Full Q see in slip)

```
import sympy as sp

x = sp.symbols('x')
f_expr = 2*x**3 - 3*x - 6
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = 2.0
tol = 0.5e-5
max_iter = 15

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break

print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

Iteration 1: $x = 1.809524$
Iteration 2: $x = 1.784200$

Iteration 3: $x = 1.783769$

Iteration 4: $x = 1.783769$

Approximate root = 1.78377 (after 4 iterations)

Q.2. Attempt any two of the following

1) Newton's divided difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = np.array([0, 1, 4, 5], dtype=float)
y = np.array([8, 11, 68, 123], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)
```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD	Order-3 DD
0	0.0	8.0	3.0	4.0	1.0
1	1.0	11.0	19.0	9.0	
2	4.0	68.0	55.0		
3	5.0	123.0			

2) Euler's Method: find $y(1.1)$, $y(1.2)$, $y(1.3)$ for $dy/dx = x*y$, $y(1)=5$, $h=0.1$. (Full Q see in slip)

```
def f(x, y):
    return x * y

x0, y0 = 1, 5
h = 0.1
x_end = 1.3

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y = y + h * f(x, y)
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

$x = 1.1000$, $y = 5.50000$

$x = 1.2000$, $y = 6.10500$

$x = 1.3000$, $y = 6.83760$

3) Simpson's 3/8 rule: $\int_0^6 1/(1+x^4) dx$, $n = 6$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 1 / (1 + x**4)

a, b = 0, 6
n = 6          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	1.0000
1	1.0000	0.5000
2	2.0000	0.0588
3	3.0000	0.0122
4	4.0000	0.0039
5	5.0000	0.0016
6	6.0000	0.0008

Approximate integral = 1.01929

Slip No. 11

Q.1. Attempt any one of the following

1) Newton's divided difference formula: evaluate $f(15)$. (Full Q see in slip)

```
import numpy as np

x = np.array([4, 5, 7, 10, 11, 13], dtype=float)
y = np.array([48, 100, 294, 900, 1210, 2028], dtype=float)
n = len(x)

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 15
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(15) \approx 3150.00000$

2) Regula-Falsi: root of $8x^3 - 2x - 1 = 0$ in $[0,1]$, rel.err < $1e-4$ or 20 iterations. (Full Q see in slip)

```
def f(x):
    return 8*x**3 - 2*x - 1

a, b = 0, 1
tol = 1e-4
max_iter = 20

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new

print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

Iteration 1: c = 0.166667, f(c) = -1.296296
Iteration 2: c = 0.338235, f(c) = -1.366909
Iteration 3: c = 0.480309, f(c) = -1.074171
Iteration 4: c = 0.572213, f(c) = -0.645561
Iteration 5: c = 0.621129, f(c) = -0.325196
Iteration 6: c = 0.644266, f(c) = -0.149163

```

Iteration 7: c = 0.654571, f(c) = -0.065464
Iteration 8: c = 0.659035, f(c) = -0.028173
Iteration 9: c = 0.660946, f(c) = -0.012022
Iteration 10: c = 0.661759, f(c) = -0.005111
Iteration 11: c = 0.662104, f(c) = -0.002170
Iteration 12: c = 0.662251, f(c) = -0.000921
Iteration 13: c = 0.662313, f(c) = -0.000390

```

Approximate root = 0.66231 (after 13 iterations)

Q.2. Attempt any two of the following

1) Newton's forward difference table . (Full Q see in slip)

```

import numpy as np
import pandas as pd

x = [2, 3, 4, 5]
y = [8, 27, 64, 125]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = D[i+1, j-1] - D[i, j-1]

columns = ['y'] + [f'Δ^{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Δ^{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Forward Difference Table:\n")
print(df)

```

Output:

Newton's Forward Difference Table:

	x	y	Δ ¹ y	Δ ² y	Δ ³ y
0	2	8	19.0	18.0	6.0
1	3	27	37.0	24.0	
2	4	64	61.0		
3	5	125			

2) RK4 Method: find y(0.2) and y(0.4) for $dy/dx = -2x*y^2$, $y(0)=1$, $h=0.2$. (Full Q see in slip)

```

def f(x, y):
    return -2 * x * y**2

x0, y0 = 0, 1
h = 0.2
x_end = 0.4

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)

```

```

k4 = h * f(x + h, y + k3)
y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
x = x + h
print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.2000, y = 0.96153
x = 0.4000, y = 0.86205

```

3) Simpson's 1/3 rule: $\int_0^6 \frac{1}{(1+x^3)} dx$, $n = 6$. (Full Q see in slip)

```

import numpy as np

def f(x):
    return 1 / (1 + x**3)

a, b = 0, 6
n = 6          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

i	xi	yi
0	0.0000	1.0000
1	1.0000	0.5000
2	2.0000	0.1111
3	3.0000	0.0357
4	4.0000	0.0154
5	5.0000	0.0079
6	6.0000	0.0046

Approximate integral = 1.14407

Slip No. 12

Q.1. Attempt any one of the following

1) Bisection: root of $x^2 + 2x - 1 = 0$ in $[0,1]$, $\text{rel.err} < 1e-4$ or 20 iterations . (Full Q see in slip)

```
def f(x):
    return x**2 + 2*x - 1

a, b = 0, 1
tol = 1e-4
max_iter = 20

if f(a) * f(b) > 0:
    print("f(a) and f(b) have the same sign; choose a different interval.")
else:
    c = a
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
        if abs(f(c)) < tol or (b - a) / 2 < tol:
            break
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 0.500000, f(c) = 0.250000
Iteration 2: c = 0.250000, f(c) = -0.437500
Iteration 3: c = 0.375000, f(c) = -0.109375
Iteration 4: c = 0.437500, f(c) = 0.066406
Iteration 5: c = 0.406250, f(c) = -0.022461
Iteration 6: c = 0.421875, f(c) = 0.021729
Iteration 7: c = 0.414062, f(c) = -0.000427
Iteration 8: c = 0.417969, f(c) = 0.010635
Iteration 9: c = 0.416016, f(c) = 0.005100
Iteration 10: c = 0.415039, f(c) = 0.002336
Iteration 11: c = 0.414551, f(c) = 0.000954
Iteration 12: c = 0.414307, f(c) = 0.000263
Iteration 13: c = 0.414185, f(c) = -0.000082
```

Approximate root = 0.41418 (after 13 iterations)

2) Newton's backward interpolation formula: evaluate $f(80)$. (Full Q see in slip)

```
import numpy as np

x = np.array([19, 39, 59, 79, 99], dtype=float)
y = np.array([41, 103, 168, 218, 235], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 80
```

```

p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
    fact *= j
    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(80) ≈ 219.78337
```

Q.2. Attempt any two of the following

1) Lagrange interpolation: find $f(10)$ from $f(5)=12$, $f(6)=13$, $f(9)=14$, $f(11)=16$. (Full Q see in slip)

```

import numpy as np

x = np.array([5, 6, 9, 11], dtype=float)
y = np.array([12, 13, 14, 16], dtype=float)
n = len(x)

xp = 10
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(10) ≈ 14.66667
```

2) Euler's Modified Method: find $y(0.1)$ and $y(0.2)$ for $dy/dx = 1 - y$, $y(0)=0$, $h=0.1$. (Full Q see in slip)

```

def f(x, y):
    return 1 - y

x0, y0 = 0, 0
h = 0.1
x_end = 0.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.1000, y = 0.09500
x = 0.2000, y = 0.18097

```

3) Simpson's 3/8 rule: $\int_4^{5.2} \ln(x) dx$, $n = 6$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return np.log(x)

a, b = 4, 5.2
n = 6          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	4.0000	1.3863
1	4.2000	1.4351
2	4.4000	1.4816
3	4.6000	1.5261
4	4.8000	1.5686
5	5.0000	1.6094
6	5.2000	1.6487

Approximate integral = 1.82785

Slip No. 13

Q.1. Attempt any one of the following

1) Newton-Raphson: root of $x^3 - 2x^2 - 4 = 0$ in $[2,3]$, $\text{rel.err} < 1e-5$ or 20 iterations . (Full Q see in slip)

```
import sympy as sp

x = sp.symbols('x')
f_expr = x**3 - 2*x**2 - 4
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = 2.5
tol = 1e-5
max_iter = 20

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break
print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: x = 2.600000
Iteration 2: x = 2.594332
Iteration 3: x = 2.594313
```

Approximate root = 2.59431 (after 3 iterations)

2) forward interpolation formula: evaluate $f(45)$. (Full Q see in slip)

```
import numpy as np

x = np.array([40, 50, 60, 70, 80], dtype=float)
y = np.array([35, 83, 153, 193, 215], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 45
p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact
```

```
print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(45) ≈ 50.50000
```

Q.2. Attempt any two of the following

1) Newton's divided difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = np.array([5, 6, 9, 11], dtype=float)
y = np.array([12, 13, 14, 16], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)
```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD	Order-3 DD
0	5.0	12.0	1.0	-0.166667	0.05
1	6.0	13.0	0.333333	0.133333	
2	9.0	14.0	1.0		
3	11.0	16.0			

2) RK2 (Heun) Method: find $y(1.1)$ and $y(1.2)$ for $dy/dx = x + y^2$, $y(1)=1$, $h=0.1$. (Full Q see in slip)

```
def f(x, y):
    return x + y**2

x0, y0 = 1, 1
h = 0.1
x_end = 1.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)
    y = y + (k1 + k2) / 2
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 1.1000, y = 1.22700
x = 1.2000, y = 1.52792
```

3) Trapezoidal rule: $\int_0^4 (x^2 + 2x + 9) dx$, $n = 8$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return x**2 + 2*x + 9

a, b = 0, 4
n = 8
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	9.0000
1	0.5000	10.2500
2	1.0000	12.0000
3	1.5000	14.2500
4	2.0000	17.0000
5	2.5000	20.2500
6	3.0000	24.0000
7	3.5000	28.2500
8	4.0000	33.0000

Approximate integral = 73.50000

Slip No. 14

Q.1. Attempt any one of the following

1) Bisection: root of $x^3 - 5x + 1 = 0$ in $[2,3]$, $\text{rel.err} < 1e-4$ or 25 iterations . (Full Q see in slip)

```
def f(x):
    return x**3 - 5*x + 1

a, b = 2, 3
tol = 1e-4
max_iter = 25

if f(a) * f(b) > 0:
    print("f(a) and f(b) have the same sign; choose a different interval.")
else:
    c = a
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
        if abs(f(c)) < tol or (b - a) / 2 < tol:
            break
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 2.500000, f(c) = 4.125000
Iteration 2: c = 2.250000, f(c) = 1.140625
Iteration 3: c = 2.125000, f(c) = -0.029297
Iteration 4: c = 2.187500, f(c) = 0.530029
Iteration 5: c = 2.156250, f(c) = 0.244049
Iteration 6: c = 2.140625, f(c) = 0.105808
Iteration 7: c = 2.132812, f(c) = 0.037865
Iteration 8: c = 2.128906, f(c) = 0.004187
Iteration 9: c = 2.126953, f(c) = -0.012579
Iteration 10: c = 2.127930, f(c) = -0.004202
Iteration 11: c = 2.128418, f(c) = -0.000009
```

Approximate root = 2.12842 (after 11 iterations)

2) Lagrange interpolation: evaluate $f(35)$ from $f(25)=52$, $f(30)=67$, $f(40)=84$, $f(50)=94$. (Full Q see in slip)

```
import numpy as np

x = np.array([25, 30, 40, 50], dtype=float)
y = np.array([52, 67, 84, 94], dtype=float)
n = len(x)

xp = 35
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(35) ≈ 77.15000
```

Q.2. Attempt any two of the following**1) Newton's backward difference table . (Full Q see in slip)**

```
import numpy as np
import pandas as pd

x = [10, 12, 14, 16]
y = [25, 50, 80, 100]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(j, n):
        D[i, j] = D[i, j-1] - D[i-1, j-1]

columns = ['y'] + [f'∇^{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(j, n):
        df.loc[i, f'∇^{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Backward Difference Table:\n")
print(df)
```

Output:

```
Newton's Backward Difference Table:
```

	x	y	∇ ¹ y	∇ ² y	∇ ³ y
0	10	25			
1	12	50	25.0		
2	14	80	30.0	5.0	
3	16	100	20.0	-10.0	-15.0

2) Euler's Method: find y(0.1)...y(0.5) for dy/dx = (y-x)/(y+x), y(0)=1, h=0.1 . (Full Q see in slip)

```
def f(x, y):
    return (y - x) / (y + x)

x0, y0 = 0, 1
h = 0.1
x_end = 0.5

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y = y + h * f(x, y)
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.1000, y = 1.10000
x = 0.2000, y = 1.18333
x = 0.3000, y = 1.25442
x = 0.4000, y = 1.31582
x = 0.5000, y = 1.36919
```

3) Simpson's 1/3 rule: $\int_{-2}^2 \frac{1}{(5+2x)} dx$, $n = 8$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 1 / (5 + 2*x)

a, b = -2, 2
n = 8          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	-2.0000	1.0000
1	-1.5000	0.5000
2	-1.0000	0.3333
3	-0.5000	0.2500
4	0.0000	0.2000
5	0.5000	0.1667
6	1.0000	0.1429
7	1.5000	0.1250
8	2.0000	0.1111

Approximate integral = 1.10503

Slip No. 15

Q.1. Attempt any one of the following

1) False Position: root of $x^2 - \ln(x) - 12 = 0$ in $[3,4]$, $\text{rel.err} < 1e-4$ or 25 iterations . (Full Q see in slip)

```
def f(x):
    return x**2 - np.log(x) - 12

a, b = 3, 4
tol = 1e-4
max_iter = 25

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 3.610611, f(c) = -0.247368
Iteration 2: c = 3.644277, f(c) = -0.012402
Iteration 3: c = 3.645957, f(c) = -0.000616
Iteration 4: c = 3.646040, f(c) = -0.000031
```

Approximate root = 3.64604 (after 4 iterations)

2) Newton's forward interpolation formula: evaluate $f(48)$. (Full Q see in slip)

```
import numpy as np

x = np.array([40, 45, 50, 55, 60, 65], dtype=float)
y = np.array([210, 253, 307, 381, 413, 492], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 48
p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact
```

```
print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(48) ≈ 279.52666
```

Q.2. Attempt any two of the following

1) Newton's divided difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = np.array([0, 1, 2], dtype=float)
y = np.array([40.7, 38.1, 39.3], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)
```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD
0	0.0	40.7	-2.6	1.9
1	1.0	38.1	1.2	
2	2.0	39.3		

2) Euler's Modified Method: find $y(0.1)$ and $y(0.2)$ for $dy/dx = -y/(1+x)$, $y(0)=2$, $h=0.1$ (the slip prints " $y(3)=2$ ", which we take as an OCR slip for $y(0)=2$ since the targets $y(0.1)$, $y(0.2)$ only make sense stepping forward from $x=0$) . (Full Q see in slip)

```
def f(x, y):
    return -y / (1 + x)

x0, y0 = 0, 2
h = 0.1
x_end = 0.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 0.1000, y = 1.81818
x = 0.2000, y = 1.66667
```

3) Trapezoidal rule: $\int_{-2}^2 x/(5+2x) dx$, $n = 8$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return x / (5 + 2*x)

a, b = -2, 2
n = 8
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	-2.0000	-2.0000
1	-1.5000	-0.7500
2	-1.0000	-0.3333
3	-0.5000	-0.1250
4	0.0000	0.0000
5	0.5000	0.0833
6	1.0000	0.1429
7	1.5000	0.1875
8	2.0000	0.2222

Approximate integral = -0.84177

Slip No. 16

Q.1. Attempt any one of the following

1) Newton-Raphson: root of $x^3 + 2x^2 + 10x - 20 = 0$ in $[1,2]$, $\text{rel.err} < 1e-5$ or 20 iterations . (Full Q see in slip)

```
import sympy as sp

x = sp.symbols('x')
f_expr = x**3 + 2*x**2 + 10*x - 20
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = 1.5
tol = 1e-5
max_iter = 20

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break
print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: x = 1.373626
Iteration 2: x = 1.368815
Iteration 3: x = 1.368808
```

```
Approximate root = 1.36881 (after 3 iterations)
```

2) Newton's backward interpolation formula: evaluate $f(63)$. (Full Q see in slip)

```
import numpy as np

x = np.array([21, 31, 41, 51, 61, 71], dtype=float)
y = np.array([20, 24, 29, 36, 46, 51], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        diff[i, j] = diff[i, j - 1] - diff[i - 1, j - 1]

xp = 63
p = (xp - x[-1]) / h

result = y[-1]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p + (j - 1))
    fact *= j
```

```

    result += (p_term * diff[-1, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(63) ≈ 47.91578
```

Q.2. Attempt any two of the following .

1) Lagrange interpolation: find $f(2)$ from $f(1)=10$, $f(3)=27$, $f(4)=65$. (Full Q see in slip)

```

import numpy as np

x = np.array([1, 3, 4], dtype=float)
y = np.array([10, 27, 65], dtype=float)
n = len(x)

xp = 2
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

```
f(2) ≈ 8.66667
```

2) RK4 Method: find $y(0.2)$ and $y(0.4)$ for $dy/dx = 3x^2y^2$, $y(0)=2$, $h=0.2$. (Full Q see in slip)

```

def f(x, y):
    return 3 * x**2 * y**2

x0, y0 = 0, 2
h = 0.2
x_end = 0.4

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.2000, y = 2.03249
x = 0.4000, y = 2.29353

```

3) Simpson's 3/8 rule: $\int_0^{12} (3x^2 + 2) dx$, $n = 12$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 3*x**2 + 2

a, b = 0, 12
n = 12          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	2.0000
1	1.0000	5.0000
2	2.0000	14.0000
3	3.0000	29.0000
4	4.0000	50.0000
5	5.0000	77.0000
6	6.0000	110.0000
7	7.0000	149.0000
8	8.0000	194.0000
9	9.0000	245.0000
10	10.0000	302.0000
11	11.0000	365.0000
12	12.0000	434.0000

Approximate integral = 1752.00000

Slip No. 17

Q.1. Attempt any one of the following .

1) Regula-Falsi: root of $x^3 + x^2 - 1 = 0$ in $[0,1]$, $\text{rel.err} < 1e-5$ or 20 iterations . (Full Q see in slip)

```
def f(x):
    return x**3 + x**2 - 1

a, b = 0, 1
tol = 1e-5
max_iter = 20

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = 0.500000, f(c) = -0.625000
Iteration 2: c = 0.692308, f(c) = -0.188894
Iteration 3: c = 0.741194, f(c) = -0.043441
Iteration 4: c = 0.751969, f(c) = -0.009335
Iteration 5: c = 0.754263, f(c) = -0.001977
Iteration 6: c = 0.754748, f(c) = -0.000417
Iteration 7: c = 0.754850, f(c) = -0.000088
Iteration 8: c = 0.754872, f(c) = -0.000019
Iteration 9: c = 0.754876, f(c) = -0.000004
```

Approximate root = 0.75488 (after 9 iterations)

2) Newton's divided difference formula: evaluate $f(3)$. (Full Q see in slip)

```
import numpy as np

x = np.array([0, 1, 2, 4, 5, 6], dtype=float)
y = np.array([1, 14, 15, 5, 6, 19], dtype=float)
n = len(x)

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 3
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]
```

```
print(f"f({xp}) ≈ {result:.5f}")
```

Output:

```
f(3) ≈ 10.00000
```

Q.2. Attempt any two of the following

1) Newton's forward difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = [-1, 0, 1, 2]
y = [1, 1, 1, -3]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = D[i+1, j-1] - D[i, j-1]

columns = ['y'] + [f'Δ{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Δ{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Forward Difference Table:\n")
print(df)
```

Output:

Newton's Forward Difference Table:

	x	y	Δ ¹ y	Δ ² y	Δ ³ y
0	-1	1	0.0	0.0	-4.0
1	0	1	0.0	-4.0	
2	1	1	-4.0		
3	2	-3			

2) RK2 (Heun) Method: find y(1.1) and y(1.2) for $dy/dx = (x^2+y^2)/x$, $y(1)=2$, $h=0.1$.(Full Q see in slip)

```
def f(x, y):
    return (x**2 + y**2) / x

x0, y0 = 1, 2
h = 0.1
x_end = 1.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h, y + k1)
    y = y + (k1 + k2) / 2
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

```
x = 1.1000, y = 2.58909
x = 1.2000, y = 3.46488
```

3) Trapezoidal rule: $\int_0^{0.5} \frac{1}{\sqrt{1-x^2}} dx$, $n = 5$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 1 / np.sqrt(1 - x**2)

a, b = 0, 0.5
n = 5
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += 2 * y[i]
integral *= (h / 2)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	1.0000
1	0.1000	1.0050
2	0.2000	1.0206
3	0.3000	1.0483
4	0.4000	1.0911
5	0.5000	1.1547

Approximate integral = 0.52424

Slip No. 18

Q.1. Attempt any one of the following

1) Newton-Raphson root of $x^3 + 7x^2 + 9 = 0$ in $[-8, -7]$, $\text{rel.err} < 1e-5$ or 20 iterations . (Full Q see in slip)

```
import sympy as sp

x = sp.symbols('x')
f_expr = x**3 + 7*x**2 + 9
f = sp.lambdify(x, f_expr, 'math')
fprime = sp.lambdify(x, sp.diff(f_expr, x), 'math')

x0 = -7.5
tol = 1e-5
max_iter = 20

xi = x0
x_next = xi
for i in range(1, max_iter + 1):
    x_next = xi - f(xi) / fprime(xi)
    rel_err = abs((x_next - xi) / x_next) if x_next != 0 else abs(x_next - xi)
    print(f"Iteration {i}: x = {x_next:.6f}")
    xi = x_next
    if rel_err < tol:
        break
print(f"\nApproximate root = {xi:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: x = -7.200000
Iteration 2: x = -7.175000
Iteration 3: x = -7.174831
Iteration 4: x = -7.174831
```

Approximate root = -7.17483 (after 4 iterations)

2) Lagrange interpolation: evaluate $f(0)$ from $f(-1)=-1$, $f(-2)=-9$, $f(2)=11$, $f(4)=69$. (Full Q see in slip)

```
import numpy as np

x = np.array([-1, -2, 2, 4], dtype=float)
y = np.array([-1, -9, 11, 69], dtype=float)
n = len(x)

xp = 0
result = 0
for i in range(n):
    term = y[i]
    for j in range(n):
        if j != i:
            term *= (xp - x[j]) / (x[i] - x[j])
    result += term

print(f"f({xp}) ≈ {result:.5f}")
```

Output:

$f(0) \approx 1.00000$

Q.2. Attempt any two of the following

1) Newton's backward difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = [1, 2, 3, 4]
y = [-1, -1, 1, 5]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(j, n):
        D[i, j] = D[i, j-1] - D[i-1, j-1]

columns = ['y'] + [f'∇{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(j, n):
        df.loc[i, f'∇{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Backward Difference Table:\n")
print(df)
```

Output:

Newton's Backward Difference Table:

	x	y	∇ ¹ y	∇ ² y	∇ ³ y
0	1	-1			
1	2	-1	0.0		
2	3	1	2.0	2.0	
3	4	5	4.0	2.0	0.0

2) RK4 Method: find $y(0.2)$ and $y(0.4)$ for $dy/dx = x^2*y$ (simplified from x^2*y^2/y), $y(0)=2$, $h=0.2$.
(Full Q see in slip)

```
def f(x, y):
    return x**2 * y

x0, y0 = 0, 2
h = 0.2
x_end = 0.4

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

x = 0.2000, y = 2.00534
x = 0.4000, y = 2.04312

3) Simpson's 3/8 rule: $\int_0^6 \frac{1}{(3x^2+2)} dx$, $n = 12$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return 1 / (3*x**2 + 2)

a, b = 0, 6
n = 12          # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	0.5000
1	0.5000	0.3636
2	1.0000	0.2000
3	1.5000	0.1143
4	2.0000	0.0714
5	2.5000	0.0482
6	3.0000	0.0345
7	3.5000	0.0258
8	4.0000	0.0200
9	4.5000	0.0159
10	5.0000	0.0130
11	5.5000	0.0108
12	6.0000	0.0091

Approximate integral = 0.58069

Slip No. 19

Q.1. Attempt any one of the following

1) Bisection: root of $x^2 + 2x - 1 = 0$ in $[-3, -2]$, $\text{rel.err} < 1e-4$ or 20 iterations . (Full Q see in slip)

```
def f(x):
    return x**2 + 2*x - 1

a, b = -3, -2
tol = 1e-4
max_iter = 20

if f(a) * f(b) > 0:
    print("f(a) and f(b) have the same sign; choose a different interval.")
else:
    c = a
    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        print(f"Iteration {i}: c = {c:.6f}, f(c) = {f(c):.6f}")
        if abs(f(c)) < tol or (b - a) / 2 < tol:
            break
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    print(f"\nApproximate root = {c:.5f} (after {i} iterations)")
```

Output:

```
Iteration 1: c = -2.500000, f(c) = 0.250000
Iteration 2: c = -2.250000, f(c) = -0.437500
Iteration 3: c = -2.375000, f(c) = -0.109375
Iteration 4: c = -2.437500, f(c) = 0.066406
Iteration 5: c = -2.406250, f(c) = -0.022461
Iteration 6: c = -2.421875, f(c) = 0.021729
Iteration 7: c = -2.414062, f(c) = -0.000427
Iteration 8: c = -2.417969, f(c) = 0.010635
Iteration 9: c = -2.416016, f(c) = 0.005100
Iteration 10: c = -2.415039, f(c) = 0.002336
Iteration 11: c = -2.414551, f(c) = 0.000954
Iteration 12: c = -2.414307, f(c) = 0.000263
Iteration 13: c = -2.414185, f(c) = -0.000082
```

Approximate root = -2.41418 (after 13 iterations)

2) Newton's forward interpolation formula: evaluate $f(27)$. (Full Q see in slip)

```
import numpy as np

x = np.array([25, 30, 35, 40, 45], dtype=float)
y = np.array([50, 67, 84, 94, 101], dtype=float)
n = len(x)
h = x[1] - x[0]

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = diff[i + 1, j - 1] - diff[i, j - 1]

xp = 27
```

```

p = (xp - x[0]) / h

result = y[0]
p_term = 1
fact = 1
for j in range(1, n):
    p_term *= (p - (j - 1))
    fact *= j
    result += (p_term * diff[0, j]) / fact

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

$f(27) \approx 55.89440$

Q.2. Attempt any two of the following

1) Newton's divided difference table . (Full Q see in slip)

```

import numpy as np
import pandas as pd

x = np.array([10, 11, 14, 16], dtype=float)
y = np.array([25, 50, 80, 100], dtype=float)
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        D[i, j] = (D[i+1, j-1] - D[i, j-1]) / (x[i+j] - x[i])

columns = ['y'] + [f'Order-{j} DD' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(n - j):
        df.loc[i, f'Order-{j} DD'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Divided Difference Table:\n")
print(df)

```

Output:

Newton's Divided Difference Table:

	x	y	Order-1 DD	Order-2 DD	Order-3 DD
0	10.0	25.0	25.0	-3.75	0.625
1	11.0	50.0	10.0	0.0	
2	14.0	80.0	10.0		
3	16.0	100.0			

2) Euler's Modified Method: find $y(0.2)$ and $y(0.4)$ for $dy/dx = (y+x)/(y-x)$, $y(0)=2$, $h=0.2$.

```

def f(x, y):
    return (y + x) / (y - x)

x0, y0 = 0, 2
h = 0.2
x_end = 0.4

```

```

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    y_predict = y + h * f(x, y)
    y_correct = y + (h / 2) * (f(x, y) + f(x + h, y_predict))
    x = x + h
    y = y_correct
    print(f"x = {x:.4f}, y = {y:.5f}")

```

Output:

```

x = 0.2000, y = 2.22000
x = 0.4000, y = 2.47864

```

3) Simpson's 1/3 rule: $\int_5^{11} (5x^2 + 3x + 2) dx$, $n = 6$. (Full Q see in slip)

```

import numpy as np

def f(x):
    return 5*x**2 + 3*x + 2

a, b = 5, 11
n = 6          # must be even
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (4 if i % 2 != 0 else 2) * y[i]
integral *= (h / 3)

print(f"\nApproximate integral = {integral:.5f}")

```

Output:

```

i      xi      yi
0      5.0000    142.0000
1      6.0000    200.0000
2      7.0000    268.0000
3      8.0000    346.0000
4      9.0000    434.0000
5     10.0000    532.0000
6     11.0000    640.0000

Approximate integral = 2166.00000

```

Slip No. 20

Q.1. Attempt any one of the following

1) Regula-Falsi: root of $x^2 - x - 0.1 = 0$ in $[1, 2]$, $\text{rel.err} < 1e-5$ or 15 iterations . (Full Q see in slip)

```

def f(x):
    return x**2 - x - 0.1

```

```

a, b = 1, 2
tol = 1e-5
max_iter = 15

c = a
for i in range(1, max_iter + 1):
    c_new = (a * f(b) - b * f(a)) / (f(b) - f(a))
    rel_err = abs((c_new - c) / c_new) if c_new != 0 else abs(c_new - c)
    print(f"Iteration {i}: c = {c_new:.6f}, f(c) = {f(c_new):.6f}")
    if f(a) * f(c_new) < 0:
        b = c_new
    else:
        a = c_new
    if rel_err < tol and i > 1:
        c = c_new
        break
    c = c_new
print(f"\nApproximate root = {c:.5f} (after {i} iterations)")

```

Output:

```

Iteration 1: c = 1.050000, f(c) = -0.047500
Iteration 2: c = 1.073171, f(c) = -0.021475
Iteration 3: c = 1.083529, f(c) = -0.009493
Iteration 4: c = 1.088086, f(c) = -0.004155
Iteration 5: c = 1.090076, f(c) = -0.001811
Iteration 6: c = 1.090942, f(c) = -0.000788
Iteration 7: c = 1.091319, f(c) = -0.000342
Iteration 8: c = 1.091482, f(c) = -0.000149
Iteration 9: c = 1.091553, f(c) = -0.000065
Iteration 10: c = 1.091584, f(c) = -0.000028
Iteration 11: c = 1.091598, f(c) = -0.000012
Iteration 12: c = 1.091604, f(c) = -0.000005

```

Approximate root = 1.09160 (after 12 iterations)

2) Newton's divided difference formula: evaluate $f(7)$. (Full Q see in slip)

```

import numpy as np

x = np.array([3, 5, 8, 9, 12], dtype=float)
y = np.array([24, 120, 504, 720, 1716], dtype=float)
n = len(x)

diff = np.zeros((n, n))
diff[:, 0] = y
for j in range(1, n):
    for i in range(n - j):
        diff[i, j] = (diff[i + 1, j - 1] - diff[i, j - 1]) / (x[i + j] - x[i])

xp = 7
result = diff[0, 0]
p_term = 1
for j in range(1, n):
    p_term *= (xp - x[j - 1])
    result += p_term * diff[0, j]

print(f"f({xp}) ≈ {result:.5f}")

```

Output:

$f(7) \approx 336.00000$

Q.2. Attempt any two of the following

1) Newton's backward difference table . (Full Q see in slip)

```
import numpy as np
import pandas as pd

x = [15, 17, 19, 21]
y = [25, 50, 80, 90]
n = len(x)

D = np.zeros((n, n))
D[:, 0] = y
for j in range(1, n):
    for i in range(j, n):
        D[i, j] = D[i, j-1] - D[i-1, j-1]

columns = ['y'] + [f'∇^{j}y' for j in range(1, n)]
df = pd.DataFrame('', index=range(n), columns=columns, dtype=object)
df['y'] = y
for j in range(1, n):
    for i in range(j, n):
        df.loc[i, f'∇^{j}y'] = D[i, j]
df.insert(0, 'x', x)

print("Newton's Backward Difference Table:\n")
print(df)
```

Output:

Newton's Backward Difference Table:

	x	y	∇ ¹ y	∇ ² y	∇ ³ y
0	15	25			
1	17	50	25.0		
2	19	80	30.0	5.0	
3	21	90	10.0	-20.0	-25.0

2) RK4 Method: find y(0.1) and y(0.2) for $dy/dx = 1 + y$, $y(0)=1.1$, $h=0.1$. (Full Q see in slip)

```
def f(x, y):
    return 1 + y

x0, y0 = 0, 1.1
h = 0.1
x_end = 0.2

n_steps = round((x_end - x0) / h)
x, y = x0, y0
for i in range(1, n_steps + 1):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
    x = x + h
    print(f"x = {x:.4f}, y = {y:.5f}")
```

Output:

x = 0.1000, y = 1.32086
x = 0.2000, y = 1.56495

3) Simpson's 3/8 rule: $\int_0^6 (x^3 + 5x + 7) dx$, $n = 12$. (Full Q see in slip)

```
import numpy as np

def f(x):
    return x**3 + 5*x + 7

a, b = 0, 6
n = 12      # must be a multiple of 3
h = (b - a) / n

x = np.linspace(a, b, n + 1)
y = f(x)

print("i\txi\ttyi")
for i in range(n + 1):
    print(f"{i}\t{x[i]:.4f}\t{y[i]:.4f}")

integral = y[0] + y[-1]
for i in range(1, n):
    integral += (2 if i % 3 == 0 else 3) * y[i]
integral *= (3 * h / 8)

print(f"\nApproximate integral = {integral:.5f}")
```

Output:

i	xi	yi
0	0.0000	7.0000
1	0.5000	9.6250
2	1.0000	13.0000
3	1.5000	17.8750
4	2.0000	25.0000
5	2.5000	35.1250
6	3.0000	49.0000
7	3.5000	67.3750
8	4.0000	91.0000
9	4.5000	120.6250
10	5.0000	157.0000
11	5.5000	200.8750
12	6.0000	253.0000

- Approximate integral = 456.00000

ALL THE BEST..... 